

TD7: Ingeniería de Datos

Calidad de datos

El objetivo de esta práctica es incorporar una herramienta de calidad de datos. Utilizaremos [Great Expectations](#), una herramienta FOSS. Para probarla, usaremos un conjunto de datos sobre viajes en subte de la Ciudad de Buenos Aires.

Configurar el ambiente

1. Levantar el compose que está en el repositorio en la carpeta `practica_data_quality`.
2. Bajar los datos.

```
PAX_URL=https://cdn.buenosaires.gob.ar/datosabiertos/datasets/sbase/subte-viajes-molinetes
wget $PAX_URL/202101_PAX15min.csv $PAX_URL/202102_PAX15min.csv -P /tmp/
sed -e s/"//g -e s/;/,/g /tmp/202101_PAX15min.csv > data/2021-01.csv
sed -e s/"//g -e s/;/,/g /tmp/202102_PAX15min.csv > data/2021-02.csv
```

* para MacOS: si el comando de “sed ...” arroja error, primero ejecutar:

```
export LC_ALL=C
```

3. Crear las tablas y cargar la data del primer mes:

```
docker compose exec -it postgres psql -U catedra -f /data/tables.sql
docker compose exec -it postgres psql -U catedra -f /data/data.sql
```

4. Pueden entrar a postgres desde la terminal con:

```
docker compose exec -it postgres psql -U catedra
```

Ejercicios

Escriba las siguientes *expectations* usando el dataset configurado. ¡Aproveche el [buscador](#)! El repo tiene configurado un archivo para ir escribiendo las validaciones.

1. Valide las columnas de la tabla para que coincidan con la secuencia del archivo de prueba batch.
2. Valide la columna **pax_total** para que sea numérica.
3. Validar la columna **estacion** para que sea de tipo *string*.
4. Valide la columna **molinete** para que no tenga valores nulos.
5. Valide la columna **linea** para que solo contenga valores del formato `Linea{A,B,C,D,E,H}`.
6. Validar que los valores de la columna **pax_total** estén en el rango `[0, 200]`.
7. Asegurar que todas las líneas de metro aparecen en **linea**.
8. Asegurar que **pax_total** siempre sea mayor o igual que **pax_pagos**.



TD7: Ingeniería de Datos

Calidad de datos

9. Validar que (**fecha,desde,hasta,molinete**) sea una clave primaria válida.
10. Arme una expresión regular para comprobar el formato de **molinete**.

Las validaciones se pueden escribir como código, pero luego se guardan como parte de Great Expectations (ver archivos `gx/checkpoints/turnstiles_checkpoint.yml`, `gx/expectations/suite.json`).

Para correr la suite se puede usar `docker compose exec great_expectations poetry run python /gq/gq/codigo.py`.

Cuando se corra la suite, se pueden ver los resultados aprovechando el archivo `gx/uncommitted/data_docs/local_site/index.html`.

Bonus

1. Normalmente no queremos correr una suite de *expectations* sobre todos los datos, sino sobre un batch en específico. Para esto podemos usar *Splitters* ([ejemplo](#)). Generalmente los *Splitters* [son por fecha](#), pero en este caso cargamos las fechas como `varchar`s. Podemos utilizar [add_splitter_column_value](#). Configure un splitter para el conjunto de datos.
2. Armar un nuevo tipo de validación para el dataset y agregarla a la suite. (Ver [guía](#).)
3. Cargar el dataset del siguiente mes y validar que las *expectations* se cumplan.

